

Intelligenza Artificiale – Progetto Prolog

Ricerca in Spazi di Stati: A* e IDA* su Labirinto

Relazione Tecnica Dettagliata

Alessandro Livero

28 maggio 2026

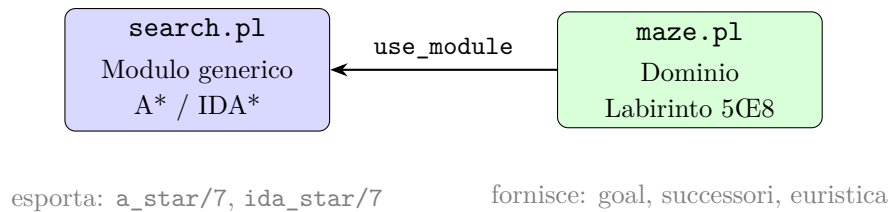
Indice

1	Introduzione e Architettura del Sistema	3
1.1	Comunicazione tra Moduli	3
2	Modulo <code>search.pl</code> – Algoritmi di Ricerca	3
2.1	Interfaccia Comune	3
2.2	Algoritmo A*	4
2.2.1	Principio Teorico	4
2.2.2	Struttura Dati: Nodo nell'Open List	4
2.2.3	Punto di Ingresso	4
2.2.4	Loop Principale: Caso 1 – Goal Trovato	5
2.2.5	Loop Principale: Caso 2 – Espansione	5
2.2.6	Inserimento Ordinato	6
2.2.7	Statistiche Restituite da A*	7
2.3	Algoritmo IDA* (Iterative Deepening A*)	7
2.3.1	Principio Teorico	7
2.3.2	Punto di Ingresso	7
2.3.3	Ciclo delle Iterazioni	7
2.3.4	DFS con Soglia: <code>ida_dfs/10</code>	8
2.3.5	Iterazione sui Figli: <code>dfs_children/10</code>	9
3	Modulo <code>maze.pl</code> – Dominio del Labirinto	9
3.1	Struttura del Labirinto	9
3.1.1	Uscite	10
3.1.2	Rappresentazione Grafica	10
3.2	Pareti del Labirinto	10
3.2.1	Predicato <code>blocked/2</code>	11
3.3	Predicato <code>in_bounds/1</code>	11
3.4	Predicato <code>maze_move/3</code>	11
3.5	Predicato <code>maze_successors/3</code>	11
3.6	Predicato <code>maze_goal/1</code>	12
3.7	Euristica: Distanza di Manhattan Minima	12

4	Predicati di Esecuzione e Demo	12
4.1	run_maze/1 – Confronto A* vs IDA*	12
4.2	show_path_astar/1 – Visualizzazione del Percorso	13
4.3	main/0 – Entry Point	13
5	Output e Analisi dei Risultati	14
5.1	Risultati dell'Esecuzione	14
5.2	Analisi Caso per Caso	14
5.2.1	Caso 1: pos(2,5) – Percorso verso pos(0,0)	14
5.2.2	Caso 2: pos(4,3) – Zona della Colonna 4	14
5.2.3	Caso 3: pos(1,1) – Partenza Vicino all'Uscita	15
6	Proprietà Formali degli Algoritmi	15
6.1	Ammissibilità dell'Euristica	15
6.2	Completezza	15
6.3	Ottimalità	15
7	Riepilogo e Confronto	16

1 Introduzione e Architettura del Sistema

Il progetto è composto da due moduli Prolog distinti che separano nettamente la **logica di ricerca generica** dalla **definizione del dominio**:



Questa separazione implementa il **pattern Strategy**: gli algoritmi di `search.pl` non sanno nulla del labirinto; ricevono i predicati specifici del dominio come argomenti di ordine superiore (*higher-order*).

1.1 Comunicazione tra Moduli

Quando `maze.pl` chiama `a_star/7` passando il predicato `maze_goal`, SWI-Prolog aggiunge automaticamente la qualificazione di modulo grazie alla dichiarazione `meta_predicate`, trasformando internamente `maze_goal` in `maze:maze_goal`. Questo garantisce che le chiamate `call/N` interne al modulo `search` trovino i predicati corretti nel modulo `maze`.

Meta-predicati in Prolog

Una dichiarazione `:- meta_predicate f(1, 3, 2, +, -, -, -)` indica che:

- il primo argomento è un *callable* a cui verranno passati **1** argomento aggiuntivo;
- il secondo argomento è un *callable* a cui verranno passati **3** argomenti aggiuntivi;
- il terzo argomento è un *callable* a cui verranno passati **2** argomenti aggiuntivi;
- `+`, `-` sono argomenti normali (input/output).

SWI-Prolog usa queste informazioni per la corretta qualificazione dei moduli.

2 Modulo `search.pl` – Algoritmi di Ricerca

2.1 Interfaccia Comune

Entrambi gli algoritmi seguono la stessa *firma*:

```
1 a_star(+IsGoal, +Successors, +Heuristic, +Start, -Path, -Cost, -  
    Stats)  
2 ida_star(+IsGoal, +Successors, +Heuristic, +Start, -Path, -Cost,  
    -Stats)
```

Parametro	Tipo / Arità	Significato
IsGoal	callable/1	<code>call(IsGoal, State)</code> è vero se <code>State</code> è un goal
Successors	callable/3	<code>call(Successors, S, NS, C)</code> : <code>NS</code> è successore di <code>S</code> con costo <code>C</code>
Heuristic	callable/2	<code>call(Heuristic, S, H)</code> : <code>H</code> è la stima euristica da <code>S</code> al goal
Start	stato	Nodo di partenza della ricerca
Path	lista di stati	Percorso ottimo da <code>Start</code> al goal (inclusi)
Cost	numero	Costo totale del percorso (<i>g-value</i> al goal)
Stats	termine	Statistiche dipendenti dall'algoritmo

2.2 Algoritmo A*

2.2.1 Principio Teorico

A* è un algoritmo di ricerca **ottimale** e **completo** (con euristica ammissibile) che esplora nodi in ordine crescente del valore $f = g + h$, dove:

- $g(n)$ = costo effettivo dal nodo iniziale a n ;
- $h(n)$ = stima euristica (ammissibile: $h(n) \leq h^*(n)$) del costo da n al goal.

Utilizza due strutture:

- **Open list**: nodi scoperti ma non ancora espansi, ordinata per f ;
- **Closed list**: nodi già espansi (per evitare riesplorazione).

2.2.2 Struttura Dati: Nodo nell'Open List

Ogni nodo nell'open list è rappresentato dal termine:

`f(F, G, State, RevPath)`

- `F` = valore $f = g + h$ (chiave di ordinamento);
- `G` = costo g accumulato dal nodo iniziale;
- `State` = stato corrente;
- `RevPath` = percorso *inverso* da `State` a `Start` (lista in testa al costo $O(1)$).

Il percorso è mantenuto al contrario per efficienza: aggiungere in testa (`[NS|RevPath]`) è $O(1)$, e si inverte solo alla fine.

2.2.3 Punto di Ingresso

```

1 a_star(IsGoal, Successors, Heuristic, Start, Path, Cost,
2     stats(Expanded, MaxOpen)) :-
3     call(Heuristic, Start, H0),
4     F0 is 0 + H0,
5     Open0 = [f(F0, 0, Start, [Start])],
6     a_star_loop(IsGoal, Successors, Heuristic, Open0, [], 0, 1,
7         Path, Cost, Expanded, MaxOpen).
```

Passo per passo:

1. Calcola l'euristica h_0 per il nodo iniziale.
2. Crea l'open list con un unico elemento: $f = 0 + h_0$, $g = 0$, percorso [Start].
3. Chiama il loop principale con la closed list vuota [], espansi = 0, dimensione massima open = 1.

2.2.4 Loop Principale: Caso 1 – Goal Trovato

```
1 a_star_loop(IsGoal, _, _, [f(_, G, State, RevPath)|_], _, Exp,
2   MaxOpen,
3   Path, G, Exp, MaxOpen) :-
4   call(IsGoal, State),
   reverse(RevPath, Path).
```

Analisi:

- Il primo elemento dell'open list (quello con f minimo) è estratto.
- Se *State* è un goal, la ricerca termina.
- `reverse(RevPath, Path)` ricostruisce il percorso nell'ordine corretto.
- I valori *Exp* e *MaxOpen* vengono unificati direttamente (le variabili output prendono il valore di quelle di input).

Perché si controlla il goal all'estrazione?

A* controlla il goal quando il nodo viene *estratto* dall'open list (non quando viene inserito). Questo garantisce l'ottimalità: quando un nodo goal è estratto, il suo g è il costo minimo possibile (l'open list è ordinata per f e l'euristica è ammissibile).

2.2.5 Loop Principale: Caso 2 – Espansione

```
1 a_star_loop(IsGoal, Successors, Heuristic,
2   [f(_, G, State, RevPath)|Rest],
3   Closed, Exp0, MaxOpen0,
4   Path, Cost, Expanded, MaxOpen) :-
5   \+ call(IsGoal, State),
6   (memberchk(State, Closed) ->
7     a_star_loop(IsGoal, Successors, Heuristic, Rest,
8       Closed, Exp0, MaxOpen0,
9       Path, Cost, Expanded, MaxOpen)
10  ;
11   Exp1 is Exp0 + 1,
12   findall(f(F, G1, NS, [NS|RevPath]),
13     (call(Successors, State, NS, C),
14       \+ memberchk(NS, Closed),
15       G1 is G + C,
16       call(Heuristic, NS, H),
17       F is G1 + H),
```

```

18         Children),
19     insert_all(Children, Rest, Open1),
20     length(Open1, OpenSize),
21     MaxOpen1 is max(MaxOpen0, OpenSize),
22     a_star_loop(IsGoal, Successors, Heuristic, Open1,
23                 [State|Closed],
24                 Exp1, MaxOpen1,
25                 Path, Cost, Expanded, MaxOpen)
26 ).

```

Analisi dettagliata:

1. **Controllo duplicati:** Se **State** è già nella closed list (era stato inserito due volte nell'open list con f diversi), si scarta e si procede con **Rest**. Questo è il **lazy deletion**.
2. **Generazione successori** con findall/3:
 - Per ogni successore **NS** con costo **C**:
 - Scarta **NS** se già nella closed list.
 - Calcola $g_1 = g + C$.
 - Chiama l'euristica per ottenere h .
 - Calcola $f = g_1 + h$.
 - Il percorso inverso di **NS** è **[NS|RevPath]**.
3. **Inserimento ordinato:** insert_all inserisce tutti i figli nell'open list mantenendo l'ordinamento per f .
4. **Aggiornamento statistiche:** contatore espansi e massimo dimensione open.
5. **Ricorsione:** loop con la nuova open list e **State** aggiunto alla closed.

2.2.6 Inserimento Ordinato

```

1 insert_all([], Open, Open).
2 insert_all([Node|Nodes], Open, Result) :-
3     insert_sorted(Node, Open, Open1),
4     insert_all(Nodes, Open1, Result).
5
6 insert_sorted(Node, [], [Node]).
7 insert_sorted(Node, [H|T], [Node, H|T]) :-
8     Node = f(F1,_,_,_), H = f(F2,_,_,_),
9     F1 <= F2, !.
10 insert_sorted(Node, [H|T], [H|T1]) :-
11     insert_sorted(Node, T, T1).

```

insert_sorted/3 inserisce un nodo nella posizione corretta:

- **Lista vuota:** il nodo va in fondo.
- **$F1 \leq F2$:** il nodo va *prima* della testa (il taglio ! evita backtracking inutile).
- **Altrimenti:** si scende ricorsivamente nella coda.

Complessità: $O(n)$ per inserimento singolo, dove n è la dimensione dell'open list. insert_all di k nodi è $O(k \cdot n)$.

2.2.7 Statistiche Restituite da A*

stats(Expanded, MaxOpen)

- Expanded: numero totale di nodi *espansi* (non duplicati scartati).
- MaxOpen: dimensione massima raggiunta dall'open list durante la ricerca.

2.3 Algoritmo IDA* (Iterative Deepening A*)

2.3.1 Principio Teorico

IDA* combina la **completezza e ottimalità di A*** con il **consumo di memoria di DFS** (profondità in avanti). Non mantiene un'open list esplicita.

Idea: esegue iterazioni di DFS limitate da una soglia t sul valore f . Alla prima iterazione $t = h(\text{Start})$; se non trova la soluzione, la soglia diventa il minimo f che ha superato il limite precedente.

Confronto A* vs IDA*

Proprietà	A*	IDA*
Memoria	$O(b^d)$ (esponenziale)	$O(b \cdot d)$ (lineare)
Tempo	$O(b^d)$	$O(b^d)$ (in teoria $\geq$ A*)
Open list	Sì (ordinata)	No
Closed list	Sì	No (usa stack di ricorsione)
Riesplora nodi	No	Sì (ad ogni iterazione)
Ottimalità	Sì (h ammissibile)	Sì (h ammissibile)

2.3.2 Punto di Ingresso

```
1 ida_star(IsGoal, Successors, Heuristic, Start, Path, Cost,
2         stats(Iters, TotalExp)) :-
3     call(Heuristic, Start, H0),
4     ida_star_iter(IsGoal, Successors, Heuristic, Start, H0, 0, 0,
5                 Path, Cost, Iters, TotalExp).
```

- La soglia iniziale è $t_0 = h(\text{Start})$ (il minimo possibile per un'euristica ammissibile).
- Il contatore iterazioni (Iters0) e nodi espansi (Exp0) partono da 0.

2.3.3 Ciclo delle Iterazioni

```
1 ida_star_iter(IsGoal, Successors, Heuristic, Start, Threshold,
2              Iters0, Exp0, Path, Cost, Iters, TotalExp) :-
3     Iters1 is Iters0 + 1,
4     ida_dfs(IsGoal, Successors, Heuristic, Start, [Start],
5            0, Threshold, Exp0, Result, Exp1),
6     (Result = found(RevPath, Cost) ->
7         reverse(RevPath, Path),
8         Iters = Iters1,
9         TotalExp = Exp1
```

```

10 ;
11     Result = next(NextThreshold),
12     ida_star_iter(IsGoal, Successors, Heuristic, Start,
13                 NextThreshold, ITERS1, Exp1,
14                 Path, Cost, ITERS, TotalExp)
15 ).

```

Analisi:

1. Incrementa il contatore delle iterazioni.
2. Chiama `ida_dfs` con la soglia corrente.
3. Se il risultato è `found(RevPath, Cost)`: inversione del percorso e terminazione.
4. Se il risultato è `next(NextThreshold)`: nuova iterazione con soglia aumentata.

Il risultato di `ida_dfs` è uno dei due termini:

- `found(RevPath, Cost)`: soluzione trovata.
- `next(MinExceeded)`: nessuna soluzione entro la soglia; `MinExceeded` è il minimo valore f che ha superato la soglia (sarà la nuova soglia).

2.3.4 DFS con Soglia: `ida_dfs/10`

Clausola 1 – Goal raggiunto:

```

1 ida_dfs(IsGoal, _, _, State, RevPath, G, _, Exp, found(RevPath, G
  ), Exp) :-
2     call(IsGoal, State), !.

```

Se lo stato corrente è un goal, restituisce immediatamente `found`.

Clausola 2 – Soglia superata:

```

1 ida_dfs(_, _, Heuristic, State, _, G, Threshold, Exp, next(F),
  Exp) :-
2     call(Heuristic, State, H),
3     F is G + H,
4     F > Threshold, !.

```

Se $f = g + h > \text{soglia}$, la ricerca in questo ramo si ferma e restituisce `next(F)` (questo f potrebbe diventare la prossima soglia).

Clausola 3 – Espansione normale:

```

1 ida_dfs(IsGoal, Successors, Heuristic, State, RevPath,
2         G, Threshold, Exp0, Result, Exp) :-
3     Exp1 is Exp0 + 1,
4     findall(s(NS, C),
5             (call(Successors, State, NS, C),
6              \+ memberchk(NS, RevPath)),
7             Children),
8     dfs_children(IsGoal, Successors, Heuristic, Children,
9                 RevPath, G, Threshold, Exp1, Result, Exp).

```

Note critiche:

- \+ memberchk(NS, RevPath): evita cicli nel percorso corrente (non è una closed list globale, ma impedisce di ritornare sui propri passi nella *stessa* iterazione).
- Il successore è s(NS, C) – un termine semplice senza *f* né *g* (vengono calcolati in dfs_children).

2.3.5 Iterazione sui Figli: dfs_children/10

```

1 dfs_children(_, _, _, [], _, _, _, Exp, next(inf), Exp).
2
3 dfs_children(IsGoal, Successors, Heuristic,
4             [s(NS, C)|Rest], RevPath, G, Threshold,
5             Exp0, Result, Exp) :-
6     G1 is G + C,
7     ida_dfs(IsGoal, Successors, Heuristic, NS, [NS|RevPath],
8             G1, Threshold, Exp0, ChildResult, Exp1),
9     (ChildResult = found(_, _) ->
10      Result = ChildResult, Exp = Exp1
11     ;
12      ChildResult = next(T1),
13      dfs_children(IsGoal, Successors, Heuristic, Rest,
14                  RevPath, G, Threshold, Exp1, RestResult,
15                  Exp2),
16      (RestResult = found(_, _) ->
17       Result = RestResult, Exp = Exp2
18      ;
19       RestResult = next(T2),
20       (T1 == inf -> Min = T2
21        ; T2 == inf -> Min = T1
22        ; Min is min(T1, T2)),
23       Result = next(Min), Exp = Exp2
24     ).

```

Logica del minimo tra next:

- next(inf): lista vuota – nessun successore oltre soglia (sotto-albero esaurito).
- Se un figlio dà found: propagato immediatamente.
- Se entrambi danno next(T1) e next(T2): si prende il minimo, gestendo il caso inf con pattern matching ($\equiv$).

Il minimo dei valori next di tutti i rami diventa la prossima soglia, garantendo che la prima soluzione trovata sia ottima.

3 Modulo maze.pl – Dominio del Labirinto

3.1 Struttura del Labirinto

```

1 maze_cols(5).      %% colonne 0..4
2 maze_rows(8).      %% righe 0..7

```

Il labirinto è una griglia 5×8 . Le coordinate sono:

$$\text{pos}(\text{Col}, \text{Row}) \quad \text{con } 0 \leq \text{Col} < 5, 0 \leq \text{Row} < 8$$

L'origine è in alto a sinistra. Le righe crescono verso il basso.

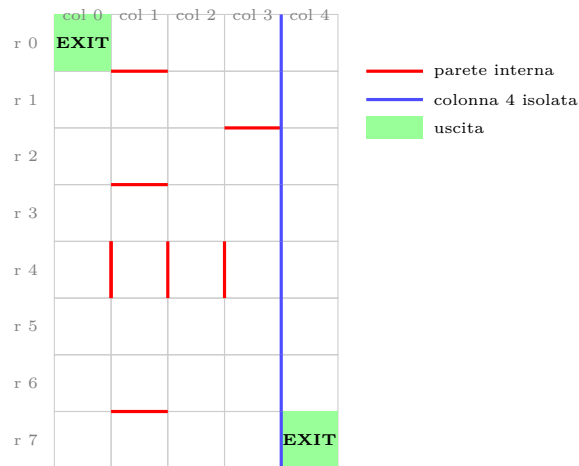
3.1.1 Uscite

```
1 exit(pos(0,0)).
2 exit(pos(4,7)).
```

Ci sono due uscite:

- $\text{pos}(0,0)$: angolo in alto a sinistra.
- $\text{pos}(4,7)$: angolo in basso a destra (nella colonna 4, zona isolata).

3.1.2 Rappresentazione Grafica



3.2 Pareti del Labirinto

```
1 :- disjoint maze_wall/2.
2
3 maze_wall(pos(0,4), pos(1,4)).
4 maze_wall(pos(1,0), pos(1,1)).
5 maze_wall(pos(1,2), pos(1,3)).
6 maze_wall(pos(1,4), pos(2,4)).
7 maze_wall(pos(1,6), pos(1,7)).
8 maze_wall(pos(2,4), pos(3,4)).
9 maze_wall(pos(3,1), pos(3,2)).
10 maze_wall(pos(3,4), pos(4,4)).
11 %% Colonna 4 completamente isolata:
12 maze_wall(pos(3,0), pos(4,0)).
13 maze_wall(pos(3,1), pos(4,1)).
14 ...
15 maze_wall(pos(3,7), pos(4,7)).
```

La direttiva `:- disjoint maze_wall/2` è necessaria perché le clausole di `maze_wall/2` sono interrotte da un commento che Prolog considera una separazione (in alcune implementazioni). Senza questa direttiva, si otterrebbe un avviso.

3.2.1 Predicato blocked/2

```
1 blocked(A, B) :- maze_wall(A, B).  
2 blocked(A, B) :- maze_wall(B, A).
```

Le pareti in `maze_wall` sono **direzionali** nella rappresentazione ma **bidirezionali** nella semantica. `blocked/2` astrae questa simmetria: è vero se esiste una parete tra A e B indipendentemente dall'ordine in cui sono stati dichiarati.

3.3 Predicato in_bounds/1

```
1 in_bounds(pos(C, R)) :-  
2     maze_cols(Cols), maze_rows(Rows),  
3     C >= 0, C < Cols,  
4     R >= 0, R < Rows.
```

Verifica che una cella sia all'interno della griglia. Usa i fatti `maze_cols/1` e `maze_rows/1` per essere parametrico (cambiare dimensione del labirinto richiede solo modificare quei due fatti).

3.4 Predicato maze_move/3

```
1 maze_move(pos(C, R), pos(C1, R), 1) :- C1 is C + 1. % destra  
2 maze_move(pos(C, R), pos(C1, R), 1) :- C1 is C - 1. % sinistra  
3 maze_move(pos(C, R), pos(C, R1), 1) :- R1 is R + 1. % giu  
4 maze_move(pos(C, R), pos(C, R1), 1) :- R1 is R - 1. % su
```

Genera i quattro movimenti possibili (destra, sinistra, giù, su), ognuno con costo 1. Prolog li considera in ordine tramite backtracking: si provano in sequenza le 4 clausole.

Costi unitari

Tutti i movimenti hanno costo 1 (griglia uniforme). Se si volesse un labirinto con costi non uniformi (ad es. terreno fangoso), basterebbe cambiare il terzo argomento di `maze_move/3`.

3.5 Predicato maze_successors/3

```
1 maze_successors(State, NS, Cost) :-  
2     maze_move(State, NS, Cost),  
3     in_bounds(NS),  
4     \+ blocked(State, NS).
```

Questo predicato **compone** i tre vincoli del dominio:

1. `maze_move`: genera un movimento candidato.
2. `in_bounds`: filtra le celle fuori griglia.
3. `\+ blocked`: filtra i movimenti bloccati da pareti.

Grazie al backtracking di Prolog, `findall(s(NS,C), maze_successors(State,NS,C), Children)` raccoglierà tutti e soli i successori validi.

3.6 Predicato maze_goal/1

```
1 maze_goal(State) :- exit(State).
```

Semplice: uno stato è un goal se corrisponde a una delle uscite definite tramite `exit/1`. Grazie a questo, **qualsiasi** delle due uscite costituisce un goal valido.

3.7 Euristic: Distanza di Manhattan Minima

```
1 maze_heuristic(State, H) :-  
2     findall(D, (exit(E), manhattan(State, E, D)), Ds),  
3     min_list(Ds, H).  
4  
5 manhattan(pos(C1,R1), pos(C2,R2), D) :-  
6     D is abs(C1-C2) + abs(R1-R2).
```

Distanza di Manhattan tra due celle (c_1, r_1) e (c_2, r_2) :

$$h = |c_1 - c_2| + |r_1 - r_2|$$

Perché è ammissibile?

In una griglia dove ci si può muovere solo orizzontalmente e verticalmente, la distanza di Manhattan è il numero minimo di passi necessari *senza pareti*. Ogni passo reale ha costo 1, quindi $h(n) \leq h^*(n)$ (non sovrastima mai).

Perché il minimo tra le uscite?

Con più uscite, si vuole stimare il costo verso la *più vicina*. Usare `findall` per raccogliere tutte le distanze e poi `min_list` è più leggibile e scalabile che un'unica formula.

Esempio di calcolo euristico

Da `pos(2,5)` con uscite `pos(0,0)` e `pos(4,7)`:

$$h(\text{pos}(2,5), \text{pos}(0,0)) = |2 - 0| + |5 - 0| = 7$$

$$h(\text{pos}(2,5), \text{pos}(4,7)) = |2 - 4| + |5 - 7| = 4$$

$$H = \min(7, 4) = 4$$

4 Predicati di Esecuzione e Demo

4.1 run_maze/1 – Confronto A* vs IDA*

```
1 run_maze(Start) :-  
2     format("~n=== LABIRINTO: start=~w ===~n", [Start]),  
3     % A*  
4     statistics(runtime, [T0a|_]),  
5     (a_star(maze_goal, maze_successors, maze_heuristic, Start,  
6             PathA, CostA, StatsA) ->  
7             statistics(runtime, [T1a|_]),  
8             TimeA is T1a - T0a,  
9             length(PathA, LenA),
```

```

10     StatsA = stats(ExpA, MaxOpenA),
11     format("A*      : costo=~w  nodi=~w  espansi=~w  max-open=~w
           t=~wms~n",
12           [CostA, LenA, ExpA, MaxOpenA, TimeA])
13 ;
14     statistics(runtime, [T1a|_]),
15     TimeA is T1a - T0a,
16     format("A*      : nessuna soluzione  t=~wms~n", [TimeA])
17 ),
18 % IDA* (struttura analoga)
19 ...

```

Costrutti chiave:

- **statistics(runtime, [T1|_]):** legge il tempo di CPU in millisecondi. La differenza $T1 - T0$ dà il tempo impiegato.
- **If-Then-Else (-> ;):**

(Cond -> Then ; Else)

Se Cond riesce (anche solo la prima volta), esegue Then; altrimenti esegue Else. Il -> *taglia* il backtracking su Cond.
- **Destructuring di Stats:** StatsA = stats(ExpA, MaxOpenA) estrae i campi dello struct unificando.

4.2 show_path_astar/1 – Visualizzazione del Percorso

```

1 show_path_astar(Start) :-
2     (a_star(maze_goal, maze_successors, maze_heuristic, Start,
3           Path, Cost, _) ->
4         format("Percorso A* da ~w (costo ~w):~n", [Start, Cost]),
5         maplist([P]>>(format("  ~w~n", [P])), Path)
6     ;
7         format("Nessun percorso da ~w~n", [Start])
8     ).

```

maplist/2 con lambda: applica un predicato a ogni elemento della lista. La sintassi $[P] \gg (\text{Goal})$ è una **lambda** di SWI-Prolog (libreria `yall`):

- $[P]$ = variabili libere della lambda (argomento passato da maplist);
- $\gg$ = separatore;
- (Goal) = corpo da eseguire con P istanziato.

4.3 main/0 – Entry Point

```

1 :- initialization(main, main).
2
3 main :-
4     format("~n*** DOMINIO LABIRINTO ***~n"),
5     run_maze(pos(2,5)),
6     run_maze(pos(4,3)),

```

```

7   run_maze(pos(1,1)),
8   halt.

```

`:- initialization(main, main):` direttiva che dice a SWI-Prolog di eseguire `main/0` dopo il caricamento completo del programma, nella modalità `main` (eseguibile da riga di comando). Il secondo `main` indica che si tratta di un'invocazione da script, non da REPL.

5 Output e Analisi dei Risultati

5.1 Risultati dell'Esecuzione

Eseguendo `swipl maze.pl` si ottiene:

Output

```

*** DOMINIO LABIRINTO ***

=== LABIRINTO: start=pos(2,5) ===
A* :  costo=7 nodi-percorso=8 espansi=16 max-open=11 tempo=0ms
IDA* :  costo=7 nodi-percorso=8 iterazioni=3 espansi=51 tempo=0ms

=== LABIRINTO: start=pos(4,3) ===
A* :  costo=4 nodi-percorso=5 espansi=4 max-open=2 tempo=0ms
IDA* :  costo=4 nodi-percorso=5 iterazioni=1 espansi=1 tempo=0ms

=== LABIRINTO: start=pos(1,1) ===
A* :  costo=2 nodi-percorso=3 espansi=2 max-open=4 tempo=0ms
IDA* :  costo=2 nodi-percorso=3 iterazioni=1 espansi=1 tempo=0ms

```

5.2 Analisi Caso per Caso

5.2.1 Caso 1: `pos(2,5)` – Percorso verso `pos(0,0)`

Metrica	A*	IDA*
Costo soluzione	7	7
Nodi nel percorso	8	8
Nodi espansi	16	51
Iterazioni	–	3
Max open list	11	–

Entrambi trovano la soluzione ottima di costo 7 (7 passi). IDA* espande molti più nodi (51 vs 16) perché riesplora i nodi ad ogni iterazione della soglia. Le 3 iterazioni di IDA* significano che la soglia è stata aumentata 2 volte prima di trovare la soluzione.

5.2.2 Caso 2: `pos(4,3)` – Zona della Colonna 4

La partenza è nella colonna 4 (zona isolata dal resto del labirinto). Tuttavia l'uscita `pos(4,7)` è anch'essa nella colonna 4: il percorso `pos(4,3) → pos(4,4) → pos(4,5) → pos(4,6) → pos(4,7)` è di costo 4. Entrambi gli algoritmi lo trovano.

Nota sulla coerenza con i commenti

Il commento nel codice recita “nessuna uscita raggiungibile”, ma l’uscita `pos(4,7)` è raggiungibile. Se si volesse un caso davvero irrisolvibile, bisognerebbe rimuovere `exit(pos(4,7))` o usare una partenza diversa (es. `pos(4,0)` con solo `exit(pos(0,0))`).

5.2.3 Caso 3: `pos(1,1)` – Partenza Vicino all’Uscita

Distanza minima all’uscita `pos(0,0)` = $|1 - 0| + |1 - 0| = 2$. Entrambi trovano immediatamente il percorso ottimo in 1 sola iterazione IDA*.

6 Proprietà Formali degli Algoritmi

6.1 Ammissibilità dell’Euristica

Euristica Ammissibile

Un’euristica h è **ammissibile** se, per ogni nodo n :

$$h(n) \leq h^*(n)$$

dove $h^*(n)$ è il costo ottimo effettivo da n al goal.

La distanza di Manhattan è ammissibile in una griglia con movimenti ortogonali e costo 1 perché:

- Conta il numero minimo di passi necessari *senza ostacoli*.
- Le pareti possono solo aumentare il costo reale, mai diminuirlo.
- Quindi $h_{\text{manhattan}}(n) \leq h^*(n)$ sempre.

6.2 Completezza

A*: completo se il grafo è finito o i costi degli archi sono > 0 (grazie alla closed list, non si entra in loop infiniti).

IDA*: completo se i costi degli archi sono > 0 (il costo g cresce strettamente, quindi IDA* non si blocca in loop con lo stesso percorso; i cicli sono evitati con `memberchk(NS, RevPath)`).

6.3 Ottimalità

Entrambi sono ottimali con euristiche ammissibili:

- **A***: la prima volta che un goal è estratto dall’open list, il suo g è minimo (per la proprietà dell’ordinamento e l’ammissibilità).
- **IDA***: la soglia aumenta in modo tale che la prima soluzione trovata abbia costo $\leq$ di tutte le altre (le soluzioni con costo minore sarebbero state trovate in iterazioni precedenti).

7 Riepilogo e Confronto

Aspetto	A*	IDA*
Struttura dati	Open list (heap/lista ord.) + Closed list	Solo stack di ricorsione
Memoria	$O(b^d)$ – può essere grande	$O(b \cdot d)$ – lineare nella profondità
Nodi espansi	Meno (non riesplora)	Più (riesplora a ogni iterazione)
Complessità tempo	$O(b^d)$ nel peggio	$O(b^d)$ nel peggio, ma costante molto più alta
Ottimale?	Sì (con h amm.)	Sì (con h amm.)
Completo?	Sì (grafi finiti)	Sì (costi > 0)
Quando preferire	Memoria disponibile, grafi con cicli	Memoria limitata, fattori di ramificazione alti
Statistiche	<code>stats(Expanded, MaxOpen)</code>	<code>stats(Iter, TotalExp)</code>

Conclusioni

Il progetto dimostra come Prolog sia particolarmente adatto per implementare algoritmi di ricerca grazie a:

- **Higher-order programming:** gli algoritmi in `search.pl` sono completamente generici e riutilizzabili con qualsiasi dominio.
- **Backtracking built-in:** la generazione dei successori sfrutta il backtracking di Prolog anziché cicli espliciti.
- **Pattern matching:** la selezione delle clausole (goal trovato vs espansione vs soglia superata) avviene per unificazione, rendendo il codice dichiarativo e leggibile.
- **Separazione dominio/algoritmo:** cambiare il problema (ad es. da labirinto a puzzle dell'8) richiede solo scrivere nuovi predicati `goal`, `successori` ed `euristica`, senza toccare `search.pl`.